



BENCHMARK NOSQL — CASSANDRA VS REDIS

Modéliser, mettre en production et comparer deux bases NoSQL sur un catalogue de films

Projet Pro final — NoSQL & Big Data

Ilham Bennebib

TMDB 5000 — 4 803 films

Les attendus

Concevoir, mettre en production et comparer deux bases NoSQL sur un vrai jeu de données .

1. Concevoir & dénormaliser

Un schéma NoSQL pensé pour les requêtes

2. Injecter un dataset réel

Exploré, nettoyé.

3. CRUD + rapport analytique

Requêtes réelles sur Cassandra et Redis

4. Administrer

Dump, restauration, performance, sécurité

TMDB 5000 : ce que j'ai trouvé, ce que j'ai corrigé

4 803 films · movies.csv (20 col.) + credits.csv (4 col., JSON imbriqué)

1 037 / 4 803

budgets à 0€ → en réalité des valeurs manquantes → converties en NaN (moyenne x2 une fois corrigée)

8 films

"Rumored" / "Post Production" → gardés dans le catalogue, exclus des classements

30 films

sans réalisateur → filtrage fait table par table (pas un filtre unique trop large)

Note pondérée (formule IMDb)

Un film à 10/10 avec 3 votes ne bat pas un film à 8/10 avec 10 000 votes.

Pourquoi Cassandra + Redis?

MongoDB

Document — connu, mais
c'était l'exemple par défaut

Cassandra

Colonnes larges — historique
durable, écriture massive



Redis

Clé-valeur — cache et
classements, latence minimale



Découvrir deux familles NoSQL différentes de MongoDB (déjà connu), et produire un vrai comparatif entre deux paradigmes opposés — écriture massive organisée par requête (Cassandra) vs accès instantané par clé (Redis) — validé par le formateur.

Une table par question, pas par entité

Principe Cassandra : lister les questions-métier d'abord, une table par requête — quitte à dupliquer les films (dénormalisation).

QUESTION-MÉTIER	TABLE	CLÉ PRIMAIRE
Fiche complète d'un film	<code>movies_by_id</code>	<code>movie_id</code>
Films les mieux notés d'un genre	<code>movies_by_genre</code>	<code>genre, weighted_rating, movie_id</code>
Films d'un réalisateur	<code>movies_by_director</code>	<code>director, release_year, movie_id</code>
Films d'un acteur	<code>movies_by_actor</code>	<code>actor_name, movie_id</code>
Films d'une année, par popularité	<code>movies_by_year</code>	<code>release_year, popularity, movie_id</code>

+ Redis : cache par film (`movie:{id}`) et classements (`leaderboard:top Rated`, `leaderboard:genre:{genre}`)

CQL pour Cassandra, des commandes pour Redis

CASSANDRA — CQL

Ressemble à SQL, se comporte différemment.

```
SELECT title, weighted_rating
FROM movies_by_genre
WHERE genre = 'Action' LIMIT 5;
```

```
SELECT AVG(weighted_rating)
FROM movies_by_director
WHERE director = 'Spielberg';
```

```
UPDATE movies_by_id SET ...
WHERE movie_id = ?;
```

REDIS — commandes

Pas de langage déclaratif : une commande par structure de donnée.

```
HSET movie:1 title "Toy Story"
      vote_average 7.7
```

```
HGETALL movie:1
```

```
ZADD leaderboard:top Rated
      8.25 862
```

```
ZREVRANGE leaderboard:top Rated
      0 9 WITHSCORES
```

Les jointures : impossibles

SQL

```
SELECT * FROM films f  
JOIN acteurs a ON ...
```

CASSANDRA / REDIS

**Aucun JOIN. On duplique les données
dans plusieurs tables/clés à l'avance.**

Dans mon projet

movies_by_id, movies_by_genre, movies_by_director, movies_by_actor, movies_by_year : le même film dupliqué 5 fois, une fois par angle de lecture.

Conséquence assumée

Aucune contrainte d'intégrité référentielle : si je modifie un titre dans une table sans le faire dans les 4 autres, Cassandra ne me préviendra jamais — c'est ma responsabilité, pas celle du moteur.

GROUP BY : oui, mais seulement sur une partition

Q Montrez-moi une requête d'agrégation

✓ Ça marche — AVG() natif, une seule partition

```
SELECT AVG(weighted_rating) FROM movies_by_director WHERE director = 'Steven Spielberg';  
→ 6.70 (sur 27 films, aucun ALLOW FILTERING nécessaire)
```

✗ Ça ne marche pas — pas de GROUP BY multi-partitions

Compter les films par année (1990-2016) : pas de requête unique possible.
release_year = clé de partition de movies_by_year → 27 requêtes, une par année.

Ce que j'en retiens : Cassandra agrège très bien dans une partition, jamais entre plusieurs. Ce qui est précalculé (ex: note moyenne par genre) l'est fait en Python avant insertion.

ACID ? Non — et c'est un choix, pas un défaut

Q Cassandra respecte-t-il ACID ?

Cassandra → BASE

- Pas de transaction multi-lignes avec rollback
- Cohérence réglable (ONE, QUORUM, ALL)
- Favorise disponibilité + tolérance réseau plutôt que cohérence stricte immédiate
- Une INSERT sur clé existante écrase silencieusement (pas d'erreur, vécu en vrai)

Redis → atomique, pas ACID

- Une commande = atomique (mono-thread)
- MULTI/EXEC groupe des commandes, mais sans rollback si ça échoue au milieu
- Durabilité dépendante de la config (RDB / AOF), pas garantie par défaut

Ne pas tourner au ralenti : index et performance

Les "index"

- Cassandra : la clé primaire EST l'index — pas de config en plus
- Index secondaire évité volontairement (anti-pattern à l'échelle)
- À la place : une table dédiée par pattern de requête
- Redis : la clé elle-même EST l'accès direct ($O(1)$)

La performance, mesurée

Séquentiel : 25-90 min

~50 000 requêtes, une par une (bloquant)

Concurrent : 1 min 28 s

`execute_concurrent_with_args`, `concurrency=100`

+ requêtes préparées (`session.prepare`) partout dans le projet

Limite assumée : `COUNT(*)` scanne toute la table — utilisé ponctuellement pour vérifier un volume, jamais en boucle.

Pannes, sauvegarde et sécurité

Q Comment gérez-vous les pannes / pertes de données ?

Dump & restauration

- Cassandra : export CSV des 5 tables (SELECT * + csv.writer)
- Restauration testée : 20 lignes réinsérées avec succès
- Redis : snapshots RDB natifs, BGSAVE à la demande

Sécurité

- Identifiants uniquement en .env, jamais dans le code
- Connexion chiffrée TLS (bundle Astra, Redis Cloud)
- Limite assumée : token Astra trop permissif (Admin)

Limite honnête à dire à l'oral

Astra ne permet pas les outils bas niveau (nodetool) — c'est une restriction du service managé, pas de Cassandra en général. Le parsing CSV des colonnes list<text> est fragile sur des noms avec virgule/apostrophe (à faire en JSON en vraie prod).

Les vraies difficultés rencontrées

Q Quelles difficultés avez-vous rencontrées ?

- | | | |
|---|---|--|
| 1 | Driver Cassandra incompatible Python 3.12 | asyncore supprimé → environnement Python 3.11 dédié créé |
| 2 | protocol_version non négocié par Astra | fixé explicitement à V4, trouvé dans la doc officielle DataStax |
| 3 | CREATE KEYSPACE interdit en CQL sur Astra | créé depuis l'interface web à la place |
| 4 | NaN pandas confondu avec du texte | "nan" littéral inséré par erreur → corrigé sur 2 champs distincts |
| 5 | INSERT sur une clé déjà existante | Cassandra écrase sans erreur (contrairement à une contrainte SQL UNIQUE) |

À dire avec assurance : ce ne sont pas des échecs, c'est la preuve que le projet a été vraiment mis en production, pas juste codé sur le papier.

Démo de l'application

CONCLUSION

Le benchmark, en une slide

	CASSANDRA	REDIS
Force principale	Historique durable, écriture massive	Latence quasi nulle, classements
Agrégation	AVG/COUNT sur 1 partition seulement	Aucune — tout est précalculé
Relations	Aucune (dénormalisation manuelle)	Aucune (structures indépendantes)
Cohérence	Réglable (tunable consistency)	Atomique par commande seulement
Cas d'usage type	Catalogue, log d'événements	Cache, leaderboard, session

Merci — questions ?